

MS4215: Advanced Data Analysis

Week 1A: Introduction to the Module

Associate Professor Sinéad Moylett

September 4, 2026

About me



Sinéad Moylett



sinead.moylett@ul.ie



Associate Professor in Statistics & Data Science, Department of Mathematics & Statistics,

UL



PhD

Trinity College Dublin,
Ireland



Postdocs + Industry

Epidemiology, public
health, neuroimmunology



Lecturer in Data Science

This module in a sentence

Developing theoretical and practical expertise in advanced data analysis, with a focus on building, interpreting, and critically evaluating statistical models.

Prerequisites: MS4213, MS4214

Basic probability. Hypothesis testing. Simple linear regression. Introductory R

Data are profoundly dumb

Judea Pearl

Weekly structure

Every week

🎓 Lecture A: 9am Monday C1059

🎓 Lecture B: 9am Thursday C1061

💻 Lab: Weeks 3-11, one per week. 10am or 5pm Tuesday C2062

💻 Lab Tutor: Nancy
(Nancy.Fosua.Boakye@ul.ie)

Your time

🕒 ~6 hours private study per week

📅 Full timetable on Brightspace

Note

Labs start **Week 3**

Get R installed before then.

Bring a laptop or use the lab machines.

Assessment

Component	Weight	When
 Online test	10%	Week 6
 Data analysis project	20%	End of Week 12
 Final exam	70%	Exam period

All components are **compulsory**. More detail on each follows.

Class rep: Deadlines



Online test (10%)

- Held in **Week 6** on Brightspace
- **Timed:** fixed window to complete
- Covers Weeks 1–5 content
- Mix of multiple choice + short answer
- Individual, open-book

Note

Think of it as a check-in: If you've kept up with lectures, it's straightforward. Only 10% of your grade.



Data analysis project (20%)

- **Individual** report on a real dataset
- Submitted as **PDF** via Brightspace
- Due: **end of Week 12**
- Covers: Sections 1 and 3
- More detail released in **Week 8**

Note

This is where you apply everything: Load data, explore it, fit models, critically evaluate results, write it up clearly.

Final exam (70%)

- **2.5 hours**, in the exam period
- All questions **compulsory**
- Every question includes **R output to interpret**
- Covers all content: Sections 1, 2, 3
- Past papers on Brightspace

Warning

You will be given **printed R output** and asked to interpret it.

No coding. Just understanding.

Brightspace and communication

On Brightspace

- Slides
- Lab sheets and datasets
- Resources and readings
- Announcements

Staying in touch

-  sinead.moylett@ul.ie
- Reply within **48 hours** weekdays within working hours
- Discussion board for questions others may share
- Announcements for module-wide updates

About these slides

Note

These slides are written in **Quarto** (`.qmd` files). Real R code runs when the slides are built, and the output appears right below it. Nothing is mocked up.

For example, this code was executed when I rendered the slides at the beginning of this lecture:

```
1 x <- c(12, 7, 9, 15, 11)
2 mean(x)
```

```
[1] 10.8
```

```
1 summary(x)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
7.0	9.0	11.0	10.8	12.0	15.0

You'll see this pattern throughout: code on the top, output on the bottom. Slides are on Brightspace so you can run the code yourself.

Tidyverse style guide



Welcome

Analyses

- 1 Files
- 2 Syntax
- 3 Functions
- 4 Pipes
- 5 ggplot2

Packages

- 6 Files
- 7 Documentation
- 8 Tests
- 9 Error messages
- 10 News

Other

- 11 Git/GitHub

Tidyverse style guide

AUTHOR

The tidyverse team

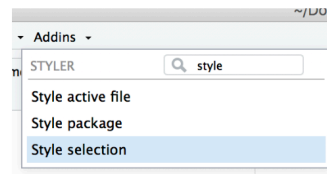
Welcome

Good coding style is like correct punctuation: you can manage without it, but it sure makes things easier to read. This site describes the style used throughout the [tidyverse](#). It was derived from Google's original R Style Guide - but Google's [current guide](#) is derived from the tidyverse style guide.

All style guides are fundamentally opinionated. Some decisions genuinely do make code easier to use (especially matching indenting to programming structure), but many decisions are arbitrary. The most important thing about a style guide is that it provides consistency, making code easier to write because you need to make fewer decisions.

Two R packages support this style guide:

- [styler](#) allows you to interactively restyle selected text, files, or entire projects. It includes an RStudio add-in, the easiest way to re-style existing code.



- [lintr](#) performs automated checks to confirm that you conform to the style guide.

Table of contents

[Welcome](#)

[Edit this page](#)

[Report an issue](#)

Generative AI policy

! AI can help you learn. It cannot do the learning for you.

You learn by **doing**. Reading the material, writing and running the code, interpreting the output, making decisions, and explaining your reasoning.

Copying and pasting an AI-generated answer might give you an answer, but it doesn't give you the **understanding** you need.

✓ Use AI to support your learning

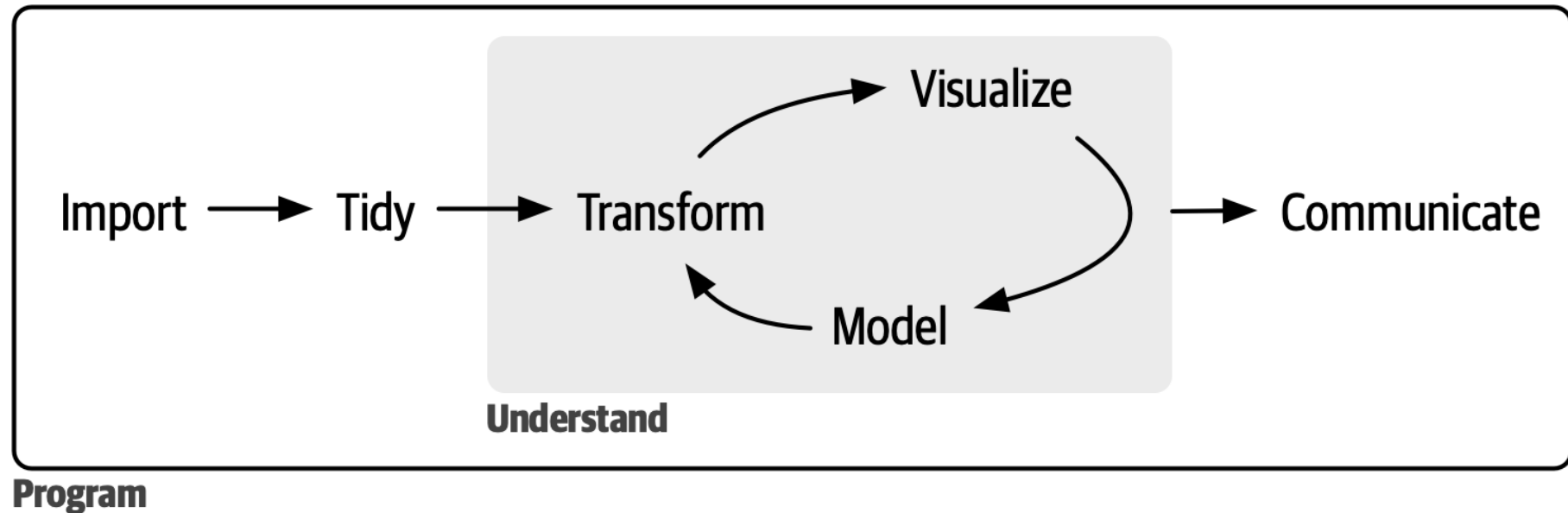
- Explain a concept in another way
- Help you debug code
- Give feedback on your understanding
- Suggest things you should check

✗ Don't outsource the learning

- Copy AI-generated analysis
- Submit AI-generated interpretations
- Have AI write your answers
- Accept an answer you cannot explain yourself

What is data analysis?

The data analysis pipeline



R for Data Science (Wickham et al.)

Every stage matters: A broken link **invalidates the conclusion**

Most time is spent on **Import + Tidy + Transform** - not modelling.

Communicate is where the value is created.

What is statistics?

“Learning from data under uncertainty.”

Statistics is the science of:

- Collecting data
- Describing data: **summarising and visualising**
- Making **inferences** about a population on the basis of a smaller sample

Data analytics and data science **cannot exist without statistics**. Every model, every inference, every measure of uncertainty is statistical.

Statistician or data scientist? The distinction isn't always clear - what you do depends heavily on the organisation you work for.

Statistics and uncertainty

Statistics provides a framework for answering questions from data in an atmosphere of **uncertainty**.

Variability and **uncertainty** are the two dominant themes:

- **Variability** we try to control, using sample size and study design
- **Uncertainty** we describe using the language of probability

These principles of statistics carry over into data science / data analytics.

A data scientist should not just give an answer, but also quantify the uncertainty in their answer.

What is data science?



Data science mainly involves processes that...

- predict
- classify
- detect
- group
- recommend

...aspects of observational data.

These processes can do this so well that sometimes we think that computers actually **understand** the data.

Sadly, this only happens in movies.

Why is data science everywhere?

The data explosion

Sensors, transactions, social media, health records: data volumes have grown exponentially

Cheap storage and compute

Cloud computing means anyone can store and process massive datasets for pennies

Every sector is data-driven

Finance, healthcare, retail, government, sport, media: all hiring people who can analyse data

Statisticians are well-placed

We understand *uncertainty*, not just patterns: that's what separates rigorous analysis from guesswork

Data → Analysis → Conclusions

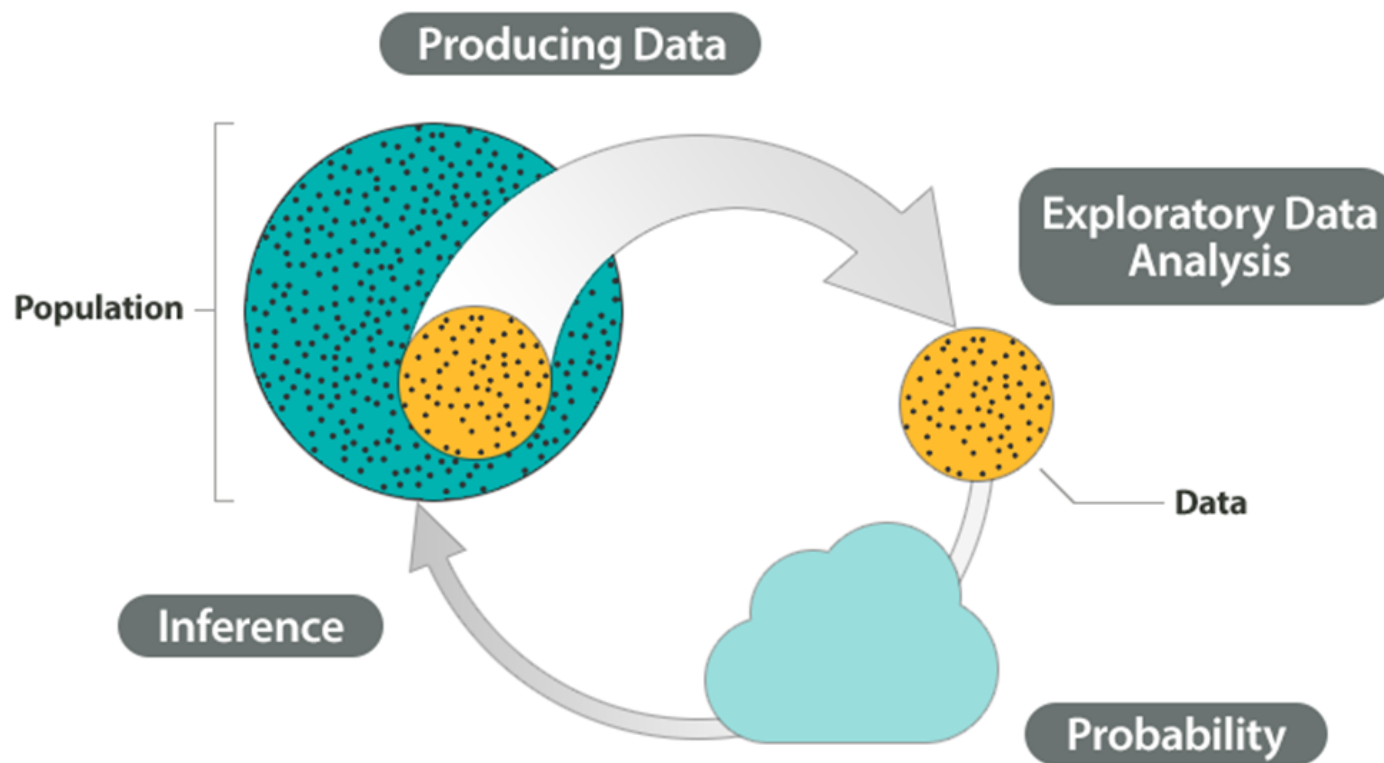
A chain with **three links**. Break any one and the conclusion is worthless.

Break point	What went wrong	Real example
 Data	Biased sample, measurement error	Early COVID fatality rate overestimate because only hospitalised patients counted
 Analysis	Wrong model, violated assumptions	Financial models pre-2008 assumed house prices never fall nationally
 Interpretation	Confusing statistical and practical significance	A treatment “works” ($p < 0.05$) but improves outcomes by 0.1%

Warning

The most dangerous output is **a confident wrong answer**.

The big picture (statistics)



Source: [The Big Picture](#)

① Producing Data

SAMPLING
It is essential that the sample is representative of the larger population.

Types of samples:
 - Volunteer Sample: almost always biased
 - Convenience Sample: at the right moment and time
 - Sampling Frame: only a selected sub-group
 - Systematic Sampling: all pop., but not completely random
 - Probability Sampling:

Simple Random Samp. **Cluster Sampling** **Stratified Sampling**

- Multistage sampling: consecutive forms of sampling

ONE VARIABLE

Point Estimation
We estimate an unknown parameter using a single number calculated from the sample data.
 $\hat{\mu}$ (mean) $\hat{\sigma}$ (std dev) $\hat{p}$ (proportion)

Interval Estimation
We estimate an unknown parameter using an interval of values likely to contain the true value, and state how confident we are this interval is correct.
 Estimating the mean μ :
 $\mu \in \hat{\mu} \pm z^* \frac{\hat{\sigma}}{\sqrt{n}}$
 Estimating the proportion p :
 $p \in \hat{p} \pm z^* \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$

Normal Distribution
t-distribution is more appropriate in cases with more variability.

Hypothesis Testing
We have some claim about the population, and we check whether or not the data obtained from the sample provides evidence against this claim. Significance level of the test α , usually 5%.

TESTING FOR p
Null hypothesis: $H_0: p = p_0$
Alternative hypothesis: $H_a: p \neq p_0$
Test statistic: $Z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$
P-value: $P(Z \geq |z|)$

TESTING FOR μ WITH σ
Null hypothesis: $H_0: \mu = \mu_0$
Alternative hypothesis: $H_a: \mu < \mu_0$ or $\mu > \mu_0$ or $\mu \neq \mu_0$
Test statistic: $Z = \frac{\bar{x} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$
P-value: $P(Z \geq |z|)$

TESTING FOR μ WITHOUT σ
Null hypothesis: $H_0: \mu = \mu_0$
Alternative hypothesis: $H_a: \mu < \mu_0$ or $\mu > \mu_0$ or $\mu \neq \mu_0$
Test statistic: $T = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$
P-value: $P(T \geq |t|)$

4 - Conclusion based on p-value
Determine if results are statistically significant (p-value $\leq \alpha$) to reject H_0 and accept H_a . If you give a 95% confidence interval for the alternative value.

② Exploratory Data Analysis

STUDY DESIGN
Observational:
 - Prospective or Retrospective: Potentially unlimited number of lurking variables make it difficult to prove causation
 - Survey: Open Q: too broad; Close Q: too narrow; Unbalanced response options; Planting ideas in Q; Complicated or sensitive Q
 - Hawthorne effect: people behave differently in an experiment than in real life.
 - memory or honesty not reliable
 - mitigate with anonymity and/or Random responses

Experiment:
 - Vocabulary: X: Factor; $X_1, X_2, \dots$: Treatments (ttt); Individuals: Subjects
 - How to reduce bias: randomly assign ttt; randomized controlled experiment; blind or double-blind; Measure outside variables with blocking ("stratification"); Matched pairs to compare responses for similar subjects

ONE VARIABLE
 Variables: v_1, v_2, v_3
 Individuals: Data
 Variable: Dataset
 1. Nominal: no ranking, just differences
 2. Ordinal: ordered differences
 3. Interval: ordinal + equal intervals
 4. Ratio: interval + meaningful 0

Population
Sampling Distribution
 Statistics vary from sample to sample due to sampling variability, and therefore can be regarded as random variables whose distribution is called sampling distribution (very theoretical).
 PARAMETER: numbers that describe a population
 STATISTIC: numbers computed from a sample
 Those are actually random variables
 $\hat{\mu}$ has a normal distribution with:
 mean of $\hat{\mu}$: $\mu_{\hat{\mu}} = \mu$
 std dev of $\hat{\mu}$: $\sigma_{\hat{\mu}} = \frac{\sigma}{\sqrt{n}}$
 $\hat{p}$ has a normal distribution with:
 mean of $\hat{p}$: $\mu_{\hat{p}} = p$
 std dev of $\hat{p}$: $\sigma_{\hat{p}} = \sqrt{\frac{p(1-p)}{n}}$
 Central Limit Theorem: Normal distribution if:
 - sample size is large enough ($n \geq 30$) OR
 - variable varies normally (or "looks normal")

TWO VARIABLES
RESPONSE (dependent) VARIABLE
 CATEGORICAL: $C \rightarrow C$
 1 - Stating the claim
 2 - Check conditions
 3 - Test statistic
 4 - Conclusion & Confidence Interval

QUALITATIVE
 1 - Stating the claim
 2 - Check conditions
 3 - Test statistic
 4 - Conclusion & Confidence Interval

PROBABILITY
 Notation: Random Experiment, Sample Space S, Events {A, B, C...}, Probability of Events: P(A), P(B), P(C)
 Venn Diagrams
 Probability Trees
 Basic Rules: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 Independence Checks: $P(A \cap B) = P(A) \cdot P(B)$
 Discrete Random Variable: assigns a unique numerical value to the outcome of a random experiment
 Continuous Random Variable: assigns a unique numerical value to the outcome of a random experiment
 Normal Distribution: μ (mean), σ (std dev)

③ Inference

Interpreting Histograms:
 Shape: symmetry / skewness and peakedness (modality)
 Center: Mean: $\bar{x} = \frac{\sum x_i}{n}$
 Spread: Standard Deviation: $s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$
 BoxPlot: illustrates 5-number summary: min, Q1, Median, Q3, max
 Standard Deviation Rule (aka Empirical Rule): 68%, 95%, 99.7%

TWO VARIABLES
RESPONSE (dependent) VARIABLE
 CATEGORICAL: $C \rightarrow C$
 1 - Stating the claim
 2 - Check conditions
 3 - Test statistic
 4 - Conclusion & Confidence Interval

QUALITATIVE
 1 - Stating the claim
 2 - Check conditions
 3 - Test statistic
 4 - Conclusion & Confidence Interval

PROBABILITY
 Notation: Random Experiment, Sample Space S, Events {A, B, C...}, Probability of Events: P(A), P(B), P(C)
 Venn Diagrams
 Probability Trees
 Basic Rules: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
 Independence Checks: $P(A \cap B) = P(A) \cdot P(B)$
 Discrete Random Variable: assigns a unique numerical value to the outcome of a random experiment
 Continuous Random Variable: assigns a unique numerical value to the outcome of a random experiment
 Normal Distribution: μ (mean), σ (std dev)

TESTING FOR p
Null hypothesis: $H_0: p = p_0$
Alternative hypothesis: $H_a: p \neq p_0$
Test statistic: $Z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$
P-value: $P(Z \geq |z|)$

TESTING FOR μ WITH σ
Null hypothesis: $H_0: \mu = \mu_0$
Alternative hypothesis: $H_a: \mu < \mu_0$ or $\mu > \mu_0$ or $\mu \neq \mu_0$
Test statistic: $Z = \frac{\bar{x} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$
P-value: $P(Z \geq |z|)$

TESTING FOR μ WITHOUT σ
Null hypothesis: $H_0: \mu = \mu_0$
Alternative hypothesis: $H_a: \mu < \mu_0$ or $\mu > \mu_0$ or $\mu \neq \mu_0$
Test statistic: $T = \frac{\bar{x} - \mu_0}{\frac{s}{\sqrt{n}}}$
P-value: $P(T \geq |t|)$

4 - Conclusion based on p-value
Determine if results are statistically significant (p-value $\leq \alpha$) to reject H_0 and accept H_a . If you give a 95% confidence interval for the alternative value.

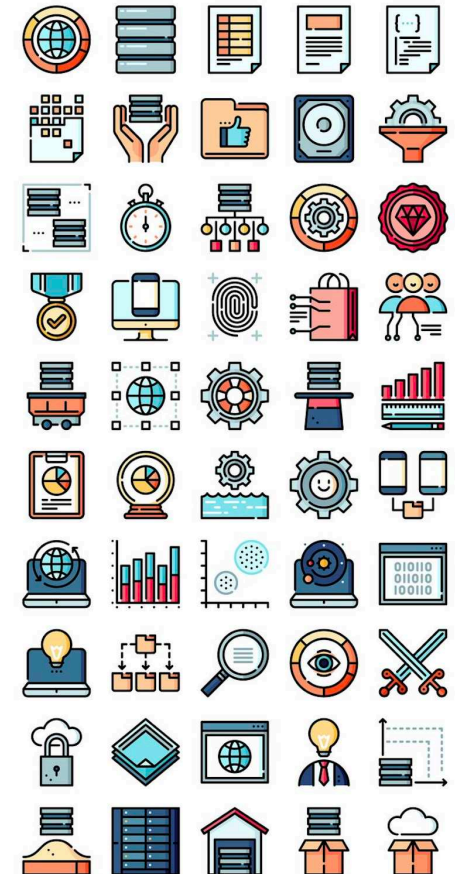
Link to scary version from Stanford Online

What is data science / data analytics?

Data science is a concept to unify:

- statistics
- data analysis
- computing and their related methods

Data science is about **gaining insight** from data so that informed decisions can be made.



What kinds of questions can we answer?

Description

What is the average rainfall in Cork?

→ Summary statistics

Prediction

What will tomorrow's temperature be?

→ Regression / machine learning

Inference

Is Limerick rainier than Dublin?

→ Hypothesis tests

Causal

Does cutting emissions reduce temperature?

→ Experimental design

MS4215 focuses most on **inference**: generalising from a sample to a population under uncertainty.

Rubbish in, rubbish out

Even a perfect model cannot rescue bad data.

Measurement error

Instrument drift, self-reported data, rounding

Selection bias

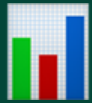
Who ended up in your sample? Are they representative?

? Missing values

Not missing at random → biased results, silently

Data entry errors

Age = 199, salary = 0, 999 coded as a value



What will we cover?

Twelve weeks, three sections

Section 1 = Weeks 1-6: Multiple Linear Regression

The foundation: one framework to rule them all

Section 2 = Weeks 7-8: ANOVA and Extensions

Groups, factors, and mixed models

Section 3 = Weeks 9-11: Generalised Linear Models

Binary, count, and non-normal outcomes

Week 12 = Revision

Consolidation and past paper practice

Each section builds on the previous.

The thread connecting them: **ANOVA is regression. Regression is a GLM**

Section 1: Multiple Linear Regression (Weeks 1–6)

Fitting and understanding

-  Simple and multiple regression
-  Interpretation of coefficients
-  Interaction terms and non-linear effects

Checking and improving

-  Assumptions and diagnostics
-  Model selection: AIC, BIC, lasso
-  Mediation and partial correlation

Note

Fitting a model takes two seconds.

Checking whether it's valid takes much longer.

Both halves matter equally.

Section 2: ANOVA and Extensions (Weeks 7–8)



One-way ANOVA

Comparing means across
two or more groups



Two-way ANOVA

Two factors and their
interaction



ANCOVA & Mixed models

Combining regression and
ANOVA; random effects

ANOVA is **not a separate topic**.

It is regression with dummy-coded categorical predictors.

Section 3: Generalised Linear Models (Weeks 9–11)

What if the outcome isn't continuous?



Logistic regression

Binary outcomes
(survived / did not)

Titanic dataset



Poisson regression

Count outcomes
(number of events)

e.g. accidents per week



Negative binomial

Over-dispersed counts
(variance > mean)

*Robust alternative to
Poisson*

Everything is a linear model

Test / model	Is actually...
t-test	ANOVA with two groups
One-way ANOVA	Regression with dummy variables
ANCOVA	Regression + categorical predictor
Logistic regression	GLM, binary outcome
Poisson regression	GLM, count outcome

**Same linear algebra runs through all of them.
One framework. One intuition. Many applications.**

What you'll be able to do



Fit and interpret

Regression models, ANOVA, and GLMs in R,
AND know what every number means



Compare and select

Use AIC, BIC, and likelihood ratio tests to
choose between models



Diagnose problems

Check assumptions, spot violations, apply
appropriate fixes



Communicate results

Write up findings clearly for a non-specialist
audience

Real data, real questions 🌐

☁️ Irish weather

2000–2023

What drives temperature and rainfall across Ireland?

Section 1: MLR

🎵 Eurovision

Song contest

Does country, year, or running order predict your score?

Section 2: ANOVA

🚢 Titanic

Passengers

What predicted survival?

Section 3: GLMs

These are real datasets - not textbook toy examples. We'll introduce the first one today.

Resources

Section 1: Regression *(free)*

 James et al. (2021), *An Introduction to Statistical Learning*

 Dalpiaz, *Applied Statistics with R*

 PSU STAT501 online course notes

Section 2: ANOVA & Mixed Models *(free)*

 Boynton, *Introduction to Statistics and Data Analysis*

Section 3: GLMs *(free)*

 Roback & Legler (2021), *Beyond Multiple Linear Regression*

 Rodríguez, *Generalized Linear Models*

Conceptual depth *(library)*

 Gelman & Hill (2020), *Regression and Other Stories*

Papers

 Wickham (2014), Tidy Data *(free)*

Read before organising your project data

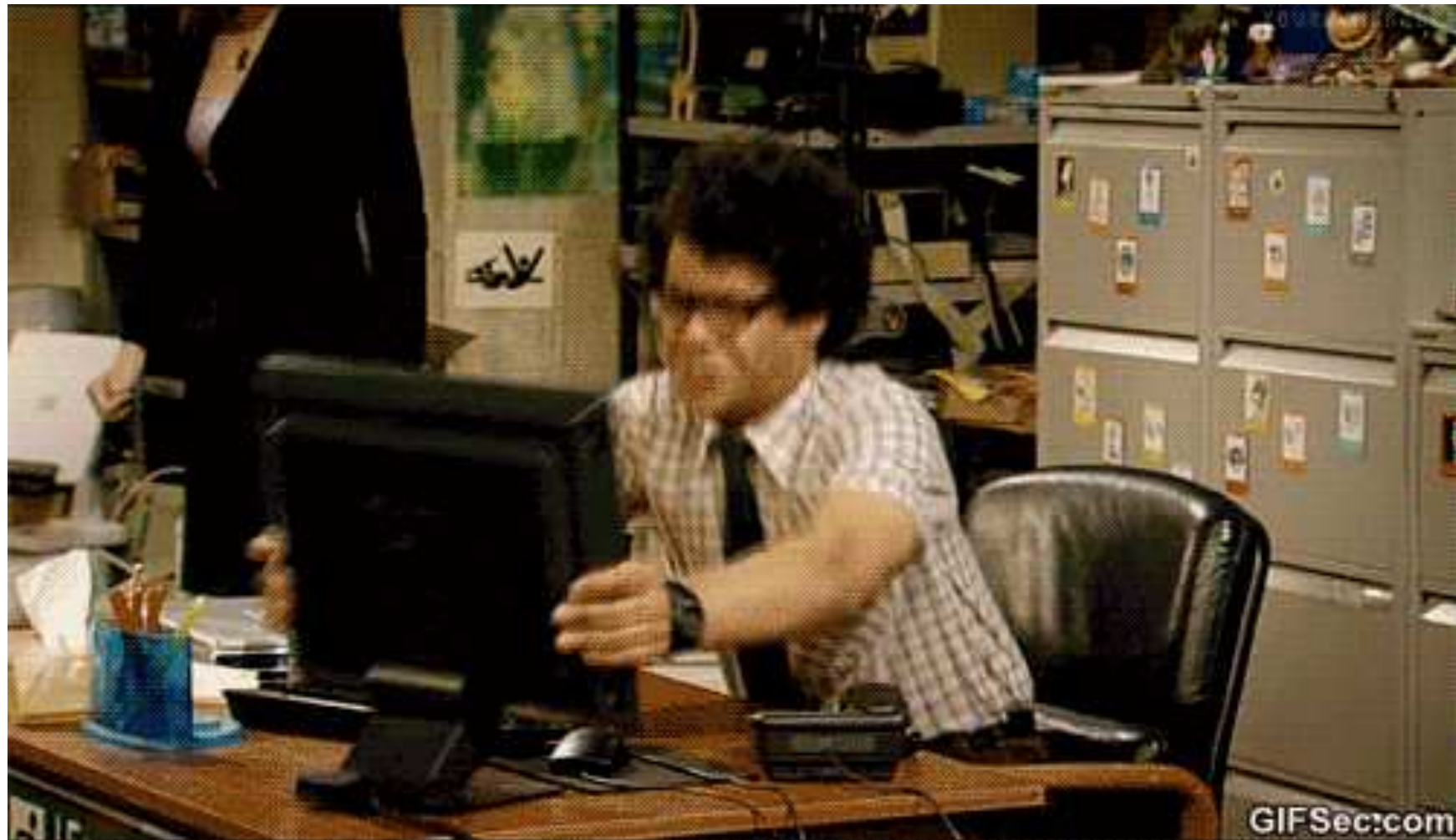
 Broman & Woo (2018), Data Organisation *(library)*

10 pages: read before Week 3

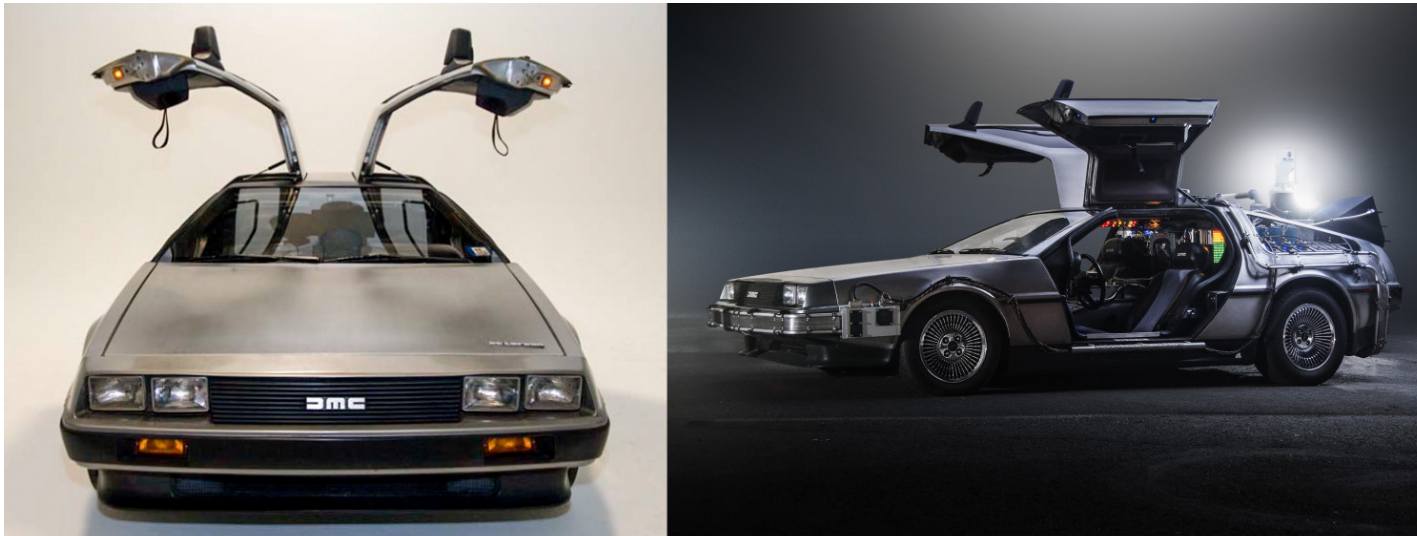


Getting set up

R versus other programming languages?



R and RStudio



R is the language

RStudio is the environment you write and run it in

Install **R first**, then **RStudio**. Both free, all platforms

Why R?



Reproducible

Your code is a permanent record of every decision



Transparent

You see exactly what the software is doing



Free

No licence: yours forever after graduation



Industry standard

The language of statistics and data science



20,000+ packages

ggplot2, tidyverse, lme4, glmnet...



Community

Stack Overflow, CRAN task views, active development

Step 1: Install R



1. Click “**Download R for [your OS]**”
2. Choose the **base** installer
3. Accept all defaults: nothing needs changing

Note

Already have R?

Check your version: type `R.version$major` in the console.

I recommend **R 4.3 or later**.

Step 2: Install RStudio

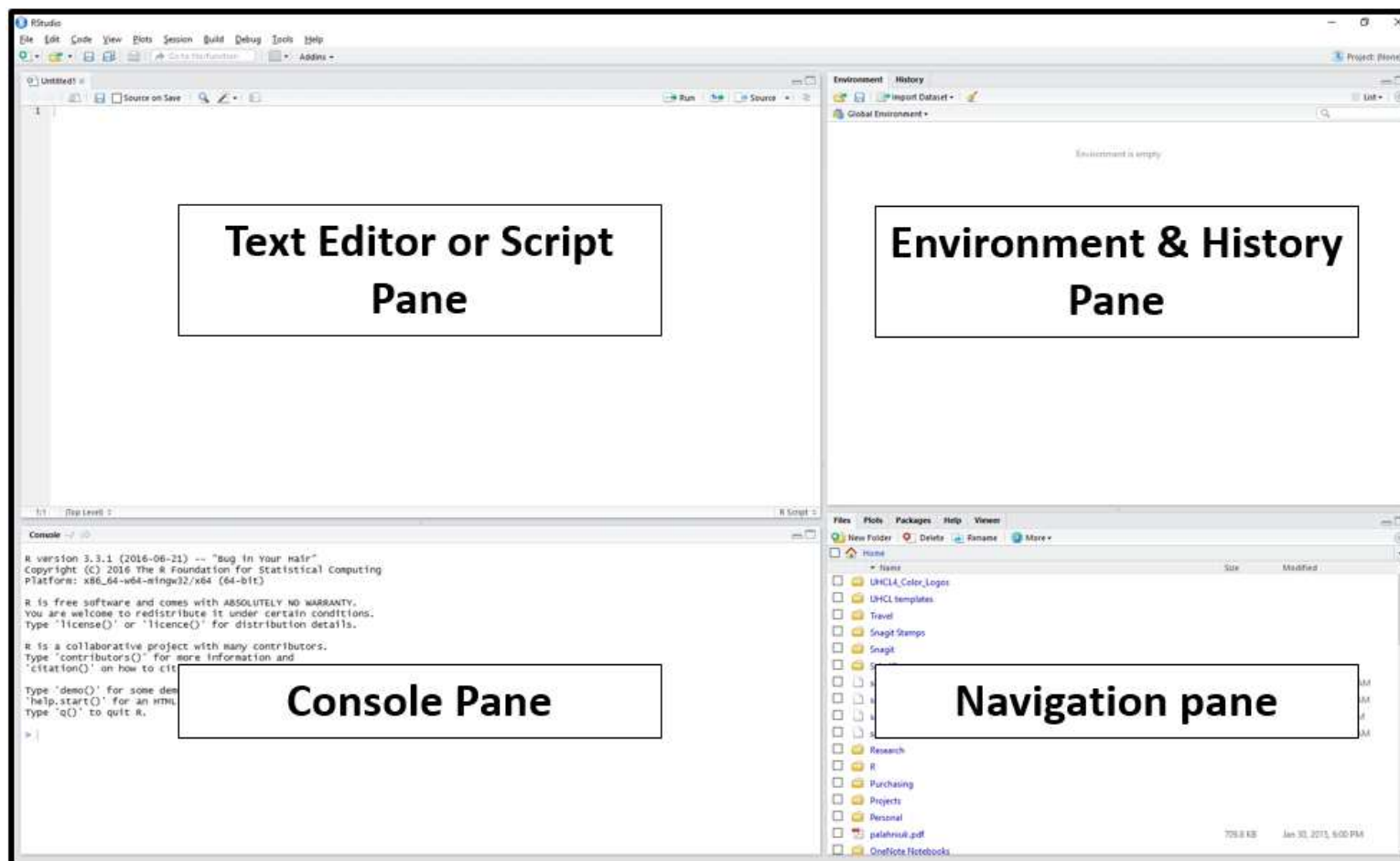
 posit.co/download/rstudio-desktop

1.  R must be installed first
2. Download **RStudio Desktop**: free edition
3. Install as normal (accept all defaults)
4. Open **RStudio**: not the plain R application

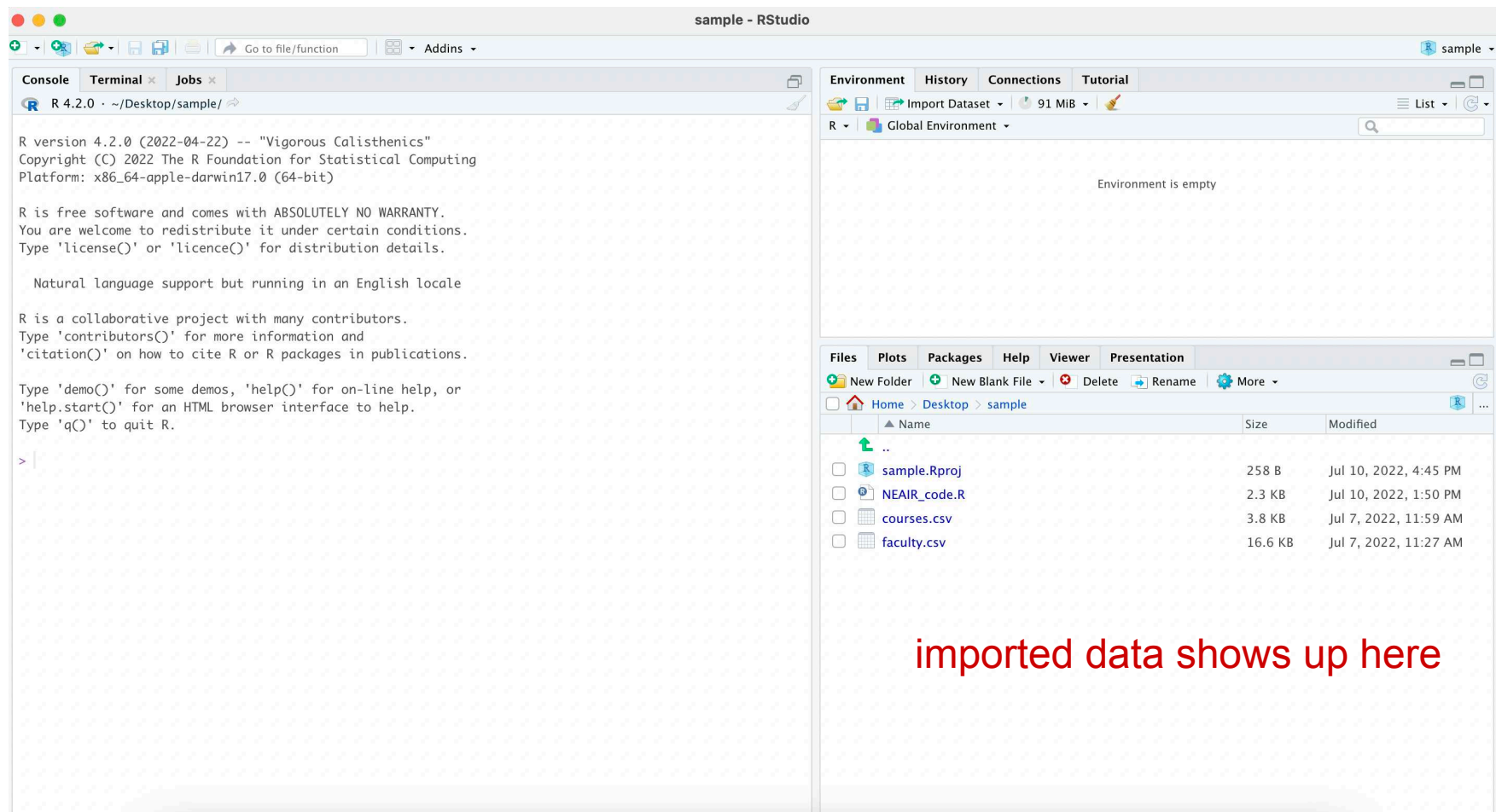
Note

Posit is the company behind RStudio. The website moved from rstudio.com to posit.co. Same product, new name.

The RStudio Interface



Navigating RStudio



code *can* go here

project files are here

Navigating RStudio

The screenshot displays the RStudio interface with the following components:

- Source Pane (Top Left):** Contains R code for loading data and filtering. The code is as follows:

```
1 library(tidyverse)
2
3 faculty <- read_csv("faculty.csv")
4 courses <- read_csv("courses.csv")
5
6 # filter----
7
8 # the `==` operator tests for equality
9
10 faculty %>%
11   filter(dept1 == "Sociology")
12
13 # the `|` operator signifies "or"
14
15 faculty %>%
16   filter(dept1 == "Sociology" |
17          dept1 == "Physics")
18
19 # the `%in%` operator allows for multiple options in a list
20
```
- Console (Bottom Left):** Shows the output of the code execution, including column specifications and data dimensions.

```
R 4.2.0 ~./Desktop/sample/
chr (4): year, rank, dept1, dept2
dbl (1): id

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> courses <- read_csv("courses.csv")
Rows: 104 Columns: 6
— Column specification —
Delimiter: ",",
chr (2): dept, level
dbl (4): semester, course_id, faculty_id, enrollment

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
>
```
- Environment Pane (Top Right):** Displays the loaded data objects.

Data	
courses	104 obs. of 6 variables
faculty	392 obs. of 5 variables
- Files Pane (Bottom Right):** Shows the project files and folders.

Name	Size	Modified
..		
sample.Rproj	258 B	Jul 10, 2022, 4:45 PM
NEAIR_code.R	373 B	Jul 10, 2022, 4:49 PM
courses.csv	3.8 KB	Jul 7, 2022, 11:59 AM
faculty.csv	16.6 KB	Jul 7, 2022, 11:27 AM

Red text annotations highlight key navigation points:

- code can also / "should" go here** (pointing to the Source pane)
- imported data shows up here** (pointing to the Environment pane)

project files are here

RStudio = Kitchen

The screenshot shows the RStudio interface with the following components and annotations:

- Source Editor:** Contains R code for loading data and filtering. A red annotation "recipe book" is placed over the code.
- Environment Panel:** Shows loaded data objects: "courses" (104 obs. of 6 variables) and "faculty" (392 obs. of 5 variables). A red annotation "ingredients" is placed over this panel.
- Files Panel:** Shows the project files: "sample.Rproj", "NEAIR_code.R", "courses.csv", and "faculty.csv". A red annotation "cupboard and refrigerator" is placed over this panel.
- Console:** Shows the output of the R code, including column specifications and data types. A red annotation "cooker" is placed over the console output.

```
1 library(tidyverse)
2
3 faculty <- read_csv("faculty.csv")
4 courses <- read_csv("courses.csv")
5
6 # filter----
7
8 # the `==` operator tests for equality
9
10 faculty %>%
11   filter(dept1 == "Sociology")
12
13 # the `|` operator signifies "or"
14
15 faculty %>%
16   filter(dept1 == "Sociology" |
17          dept1 == "Physics")
18
19 # the `%in%` operator allows for multiple options in a list
20
```

Console Output:

```
R 4.2.0 - ~/Desktop/sample/
chr (4): year, rank, dept1, dept2
dbl (1): id

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> courses <- read_csv("courses.csv")
Rows: 104 Columns: 6
— Column specification —
Delimiter: ","
chr (2): dept, level
dbl (4): semester, course_id, faculty_id, enrollment

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
>
```

Always work in a script

“If it’s not in a script, it didn’t happen.”

✗ Console only

- Output disappears when you close R
- Can’t re-run your analysis
- Can’t share or submit your work

✓ Script (.R file)

- Permanent record of every step
- Re-run with one click
- Share, submit, collaborate

Create a new script: **File → New File → R Script** (**Ctrl/Cmd + Shift + N**)

Run a line: **Ctrl + Enter** (Windows) / **Cmd + Enter** (Mac)

When learning R... 🤔

RECENTLY I DID A
DIPLOMA TO LEARN TO
CODE IN R



IT DID NOT FEEL
EASY



EXCEPT KNITTING DOESN'T
THROW UP LOADS OF ERRORS



I ALSO CAN'T KNIT ONE AS
I'M TERRIBLE AT KNITTING
[@twisteddoodles](#)

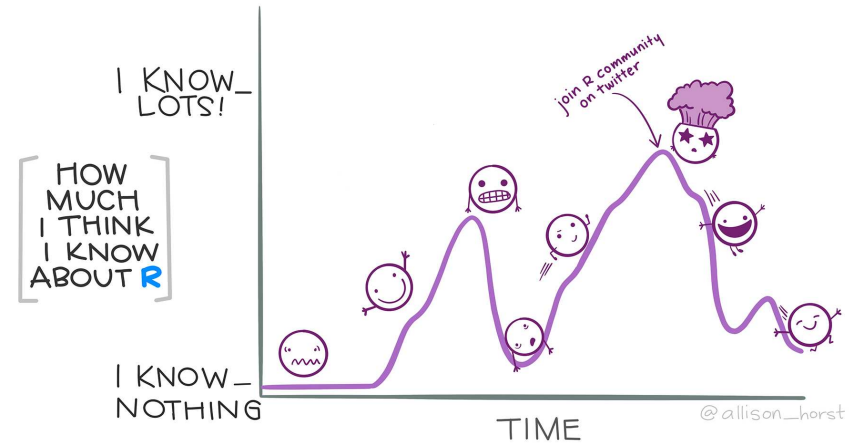
R is MEANT
TO BE ONE OF THE
EASIEST CODING
LANGUAGES



IT'S LIKE KNITTING
WITH TEXT!



AND I CAN'T CODE
SOMEONE A SCARE



Artwork by [@allison_horst](#) [@twisteddoodles](#)

The role of R

Code = reproducible analysis. Not a black box.

What the labs train

Writing and running R code: fitting models, creating plots, checking output

What the exam tests

Understanding output: What does this coefficient mean? Why does this diagnostic look wrong?

Note

In the exam you will be given **printed R output** to interpret.

You will not be typing code.

The skill is understanding - not syntax.

Getting started with data in R

Your first R commands

```
1 # Arithmetic
2 2 + 3
3 10 / 4
4 2^8
```

```
[1] 5
[1] 2.5
[1] 256
```

```
1 # Assignment
2 x <- 42
3 x
```

```
[1] 42
```

```
1 # Print explicitly
2 print("Hello, MS4215")
```

```
[1] "Hello, MS4215"
```

R objects: Vectors

```
1 # Numeric vector
2 temps <- c(12.1, 14.3, 11.8, 15.2)
3 class(temps)
4 length(temps)
5 mean(temps)
```

```
[1] "numeric"
[1] 4
[1] 13.35
```

```
1 # Character vector
2 stations <- c("Cork", "Dublin", "Shannon")
3 class(stations)
```

```
[1] "character"
```

```
1 # Logical vector (result of comparison)
2 is_warm <- temps > 13
3 is_warm
```

```
[1] FALSE TRUE FALSE TRUE
```

Three types we'll use most:

 **numeric** = continuous data  **character** = categories as text  **logical** = TRUE / FALSE

Tidy data

Messy

	Jan	Feb	Mar
Dublin	54	48	51
Cork	82	73	79

Months hidden in column headers

Tidy

city	month	rainfall
Dublin	Jan	54
Dublin	Feb	48
Cork	Jan	82
Cork	Feb	73

Each variable a column; each observation a row

Wickham (2014): Every variable is a column. Every observation is a row. Every cell holds one value

Data organisation in spreadsheets

Broman & Woo (2018): the five most common mistakes:

1.  **Merged cells** = break filtering and sorting; never merge
2.  **Colour as data** = “highlighted yellow = outlier” is invisible to R
3.  **Multiple tables per sheet** = one dataset per sheet
4.  **Inconsistent coding** = `Male`, `male`, `M`, `MALE` are four different things
5.  **Spaces in names** = use `snake_case`; `My Variable` becomes a headache

R objects: Data frames

```
1 weather <- data.frame(  
2   station    = c("Dublin", "Cork", "Shannon"),  
3   avg_temp   = c(9.8, 10.5, 9.6),  
4   annual_rain = c(841, 1594, 1180)  
5 )
```

```
1 weather
```

	station	avg_temp	annual_rain
1	Dublin	9.8	841
2	Cork	10.5	1594
3	Shannon	9.6	1180

```
1 nrow(weather)  
2 ncol(weather)
```

```
[1] 3  
[1] 3
```

This is **tidy data** - one row per station.



Rows = observations



Columns = variables

Indexing in R

```
1 # By position [row, col]
2 weather[1, ]      # row 1, all columns
3 weather[, 2]      # column 2, all rows
```

```
      station avg_temp annual_rain
1 Dublin      9.8         841
[1]  9.8 10.5  9.6
```

```
1 # By name with $
2 weather$station
3 weather$annual_rain
```

```
[1] "Dublin" "Cork"   "Shannon"
[1]  841 1594 1180
```

```
1 # Logical subset (filter rows)
2 weather[weather$annual_rain > 1000, ]
```

```
      station avg_temp annual_rain
2   Cork      10.5         1594
3 Shannon      9.6         1180
```

Installing and loading packages

```
1 # Download once (ever)
2 install.packages("ggplot2")
3
4 # Load every session
5 library(ggplot2)
6
7 # Defensive pattern for shared scripts
8 if (!requireNamespace("ggplot2",
9   quietly = TRUE)) {
10   install.packages("ggplot2")
11 }
12 library(ggplot2)
```



`install.packages()`

Download the package = **once**



`library()`

Open the package = **every session**

⚠ Don't put `install.packages()` at the top of every script! It's slow and overwrites existing installations.

Loading and checking data

```
1 # CSV
2 df <- read_csv("data/myfile.csv")
3
4 # Excel
5 library(readxl)
6 df <- read_excel("data/myfile.xlsx")
7
8 # Check where R is looking
9 getwd()
```

File paths matter

✅ Relative path: `"data/file.csv"` *(works on any computer in the same project)*

❌ Absolute path: `"C:/Users/sinead/..."` *(breaks on every other computer)*

💡 Use **RStudio Projects** to set your working directory automatically.

Exploring a new dataset

```
1 str(df)      # types and structure
2 head(df)     # first 6 rows
3 tail(df)     # last 6 rows
4 dim(df)      # rows × columns
5 names(df)    # column names
6 summary(df)  # ranges, NAs, quartiles
```

Your 6-step checklist

1. Are column types correct?
2. Any obvious coding errors?
3. Missing values (NAs)?
4. Plausible ranges?
5. How many rows do you have?
6. What do the column names mean?

Run these **before any analysis**.

Live demo: Irish weather data

```
1 weather <- read_csv("data/ireland_weather_2000_2023.csv")
```

```
1 str(weather)
```

```
spc_tbl_ [42,190 × 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ station: chr [1:42190] "Shannon" "Shannon" "Shannon" "Shannon" ...
 $ date    : Date[1:42190], format: "2000-01-01" "2000-01-04" ...
 $ tmax_c  : num [1:42190] 8.7 7.3 11 11.1 7.8 8 11.7 7.4 4.7 6 ...
 $ tmin_c  : num [1:42190] NA NA 3.9 NA NA NA 4 2.4 -1.4 -1.4 ...
 $ prcp_mm: num [1:42190] 1.5 NA 1.8 NA 1.5 NA 5.3 0.8 1.5 1.5 ...
 - attr(*, "spec")=
   .. cols(
   ..   station = col_character(),
   ..   date = col_date(format = ""),
   ..   tmax_c = col_double(),
   ..   tmin_c = col_double(),
   ..   prcp_mm = col_double()
   .. )
 - attr(*, "problems")=<pointer: 0xc4b3d9630>
```

Live demo: Irish weather data

```
1 head(weather, 3)
```

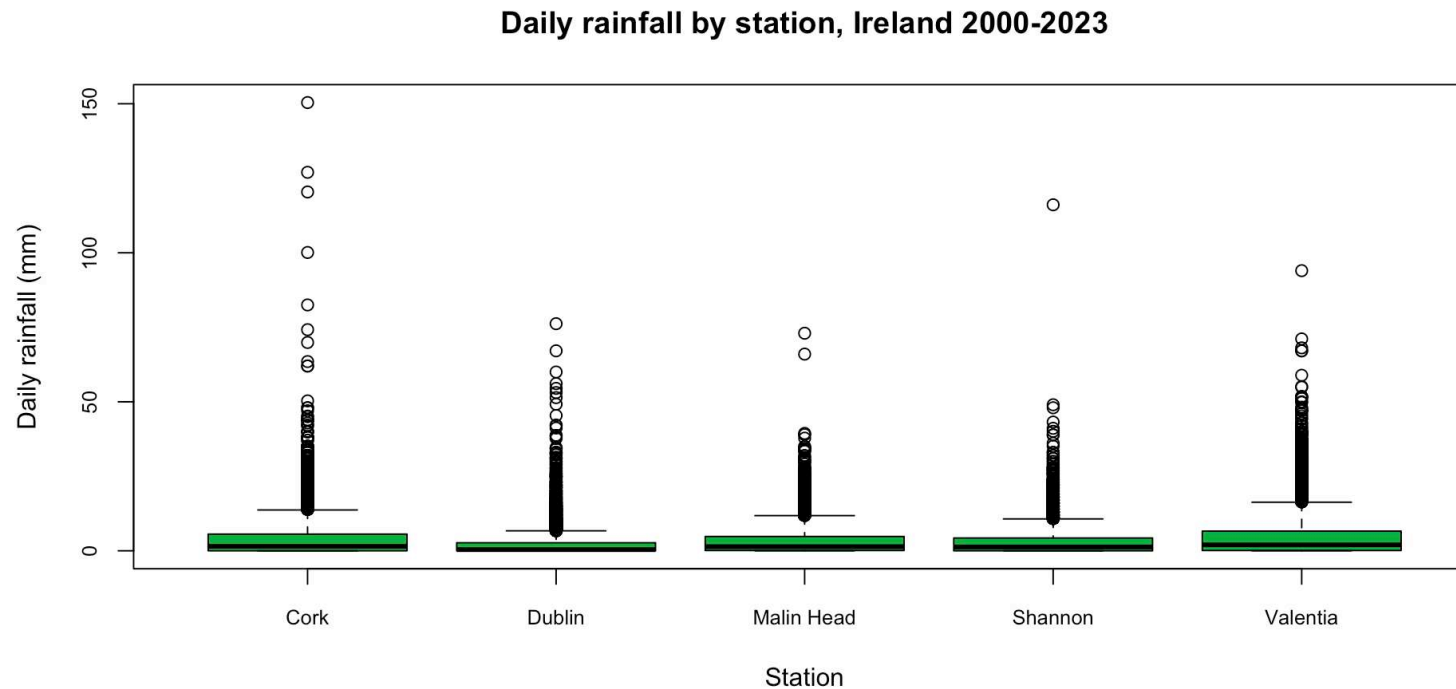
```
# A tibble: 3 × 5  
  station date      tmax_c tmin_c prcp_mm  
  <chr>   <date>    <dbl> <dbl>   <dbl>  
1 Shannon 2000-01-01     8.7    NA     1.5  
2 Shannon 2000-01-04     7.3    NA     NA  
3 Shannon 2000-01-05    11     3.9     1.8
```

```
1 summary(weather[, c("tmax_c", "prcp_mm")])
```

tmax_c		prcp_mm	
Min.	:-4.70	Min.	: 0.000
1st Qu.:	10.40	1st Qu.:	0.000
Median :	13.70	Median :	1.200
Mean :	13.72	Mean :	3.577
3rd Qu.:	17.00	3rd Qu.:	4.800
Max.	:33.00	Max.	:150.400
NAs	:10264	NAs	:5861

Is Limerick rainier than Dublin? 🌧️

```
1 boxplot(prcp_mm ~ station,  
2         data = weather,  
3         col = "#00B140",  
4         xlab = "Station",  
5         ylab = "Daily rainfall (mm)",  
6         main = "Daily rainfall by station, Ireland 2000-2023",  
7         cex.axis = 0.8)
```



Packages we'll use this term



Data & visualisation

`tidyverse` · `readxl` · `ggplot2`
· `corrplot`

Section 1 (*Regression*)

`car` · `interactions` · `glmnet` ·
`ppcor`

Section 2 (*ANOVA*)

`lme4` · `emmeans` · `afex` ·
`performance`

Section 3 (*GLMs*)

`pROC` `caret`
`ResourceSelection` `AER`

R functions

Function	Purpose
<code>install.packages("pkg")</code>	Install a package from CRAN
<code>library(pkg)</code>	Load an installed package
<code>read_csv("file.csv")</code>	Read a CSV file into a data frame
<code>str(df)</code>	Show data types and first few values
<code>summary(df)</code>	Summary statistics for each variable
<code>head(df)</code>	Show first 6 rows

Summary



Statistics is learning from data under uncertainty. The foundation of all data science / data analytics



The pipeline matters end-to-end: bad data or bad interpretation ruins good modelling



Module structure: MLR $\rightarrow$ ANOVA $\rightarrow$ GLMs - all connected through the linear model framework



R basics: vectors, data frames, indexing, loading and exploring data with `str()` and `summary()`

Before next lecture: Install R and RStudio. Install some packages.